CS4375 Fall 2025
<Gabriel Peña >
<gpena6@miners.utep.edu >

Homework Report
Submitted on <11/10/2025 >

# HW 4: Lazy Allocation for xv6

**Task 1. freepmem() system call**

For Task 1, I set up the `freepmem()` system call in xv6 by connecting it through both user and kernel space: I included the prototype in `user/user.h`, made a stub in `usys.pl`, created a `free.c` command to invoke it, and registered `_free` and `_memory-user` in the Makefile.

In the kernel, I defined `SYS_freepmem` in `syscall.h`, declared and added `sys_freepmem` to the syscall table in `syscall.c`, implemented it in `sysproc.c` to call `kfreepmem()`, and wrote `kfreepmem()` in `kalloc.c` to tally the free pages in the allocator's freelist.

Testing with the `free` command showed that the syscall accurately reported available memory, decreasing by the expected amount after allocations and returning to baseline after frees. Running `memory-user` in the background confirmed that `malloc()` fails when the requested memory exceeds the free physical memory, with failure happening when about 53 MB was free and 81 MB was requested.

Through this task, I learned how to extend xv6 with a new system call from start to finish, how its straightforward page allocator keeps track of free memory, and how user allocations interact with the kernel's freelist, giving me hands-on experience with syscall design and memory

```
$ memory-user 1 100 10 &
$ allocating 0x0000000000000001 mebibytes
malloc returned 0x0000000000003010
freeing 0x0000000000000001 mebibytes
allocating 0x000000000000000B mebibytes
malloc returned 0x0000000000103020
freeing 0x000000000000000B mebibytes
allocating 0x0000000000000015 mebibytes
malloc returned 0x0000000000C03030

$ free
Free memory: 98680832 bytes
$ freeing 0x0000000000000015 mebibytes
```

**Task 2. Change sbrk() so that it does not allocate physical memory.**

In Task 2, I tweaked the sbrk() system call in xv6 to expand the process's virtual address space without actually allocating any physical memory. I achieved this by taking out the call to growproc() and just bumping up the sz field in the process structure. Consequently, when a

process tried to access the newly allocated heap memory, xv6 triggered a page fault (usertrap() with scause = 0xf) since there were no physical pages mapped. Because xv6 lacks a page fault handler, the process ended up being terminated, and the kernel freaked out with an error like uvmunmap: not mapped. This task really showcased the divide between virtual and physical memory and underscored how crucial demand paging and fault handling are in today's operating

```
init: starting sh
$ memory-user 1 100 10 &
usertrap(): unexpected scause 0x000000000000000f pid=3
            sepc=0x00000000000012c4 stval=0x0000000000004008
panic: uvmunmap: not mapped
```

**Task 3. Handle the load and store faults that result from Task 2**

```
init: starting sh
$ memory-user 1 100 10 &
panic: uvmunmap: not mapped
```

I figured out how to catch load and store page faults in xv6, and allocate physical memory only when a process tries to access unmapped virtual pages. This experience taught me the fundamentals of demand paging and how to properly set up page table mappings, allowing the system to recover from faults rather than just crashing.

Some difficulties that I came across was making the changes and then catching those error that would happen cause of those changes. Finding where to make those extra modifications in the directory file for me to get it to work

**Task 4. Fix kernel panic and any other errors.**

```
$ allocating 0x0000000000000001 mebiby
malloc returned 0x0000000000003010
freeing 0x0000000000000001 mebibytes
allocating 0x000000000000000B mebibyte
malloc returned 0x0000000000103020
freeing 0x000000000000000B mebibytes
allocating 0x0000000000000015 mebibyte
```

I fixed some errors by updating a few kernel files. In trap.c, I included handling in usertrap() for load/store page faults (scause == 13 or 15), which involves allocating pages using kalloc() and mapping them with mappages(). Instead of panicking when allocation fails, I made it print error messages. In vm.c, I tweaked uvmcopy() to skip over unmapped pages so that fork() can work with lazy allocation, and I adjusted uvmunmap() to handle unmapped pages to avoid crashes when a process exits. Lastly, in sysproc.c, I worked on sys_sbrk()

By completing Task 4, I figured out how to enhance xv6's robustness with demand paging by making sure kernel functions like uvmcopy and uvmunmap can handle unmapped pages safely. This experience highlighted the significance of defensive programming in operating systems, where functions need to be able to deal with lazy allocation instead of just assuming that all pages are available. I faced a challenge with frequent kernel panics when processes exited or forked, as those scenarios expected fully mapped memory. I tackled this issue by adjusting both functions to gracefully skip unmapped pages and by incorporating error checks in usertrap() for failed allocations.

Task 5. Test your lazy memory allocation.
Describe the purpose of each of your test cases and show the results. Commit and push your test programs to the user directory in your hw4 branch.

Extra Credit Task 6. Enable use of the entire virtual address space.
List what files you changed and describe the changes.
Summarize what you learned by carrying out this task.

Extra Credit Task 7. Allow a process to turn memory overcommitment on and off.
Describe your approach to this task. Commit and push your code changes to your hw4 branch.
Show the results of a test program that turns memory overcommitment on and off.
Summarize what you learned by carrying out this task.